

1. We convert raw audio into spectrogram or MFCC-style features because speech patterns are much easier to recognize in the frequency domain than in a raw waveform. A waveform is just a stream of amplitude values, while a spectrogram or MFCC shows how the important frequencies change over time. That gives the keyword spotting model a cleaner, more informative input and makes learning “YES” versus “NO” much easier.
2. The representative dataset is used to help the converter figure out the real value ranges that appear inside the model during inference. Those ranges are needed to choose the INT8 scaling factors and zero-points when converting from float to integer. If the representative data is not realistic, the quantized model can lose accuracy because the numeric ranges will be calibrated poorly.
3. `audio_processor.get_data()` loads audio samples from the dataset and prepares them in the exact format the model expects. It can apply preprocessing like time shifting, adding background noise, and generating the final feature “fingerprint” used for training or testing. It then returns both the processed input data and the matching label indexes.
4. `g_model` is the C/C++ byte array that stores the quantized TensorFlow Lite model, and `g_model_len` is the number of bytes in that array. In my exported file, `g_model_len` is `18984`, so the Arduino code needs that exact size to read the model correctly. If either one is copied incorrectly, the board may fail to load the model or run broken inference.

```
Heard yes (201) @26480ms
Heard unknown (201) @29264ms
Heard yes (209) @31504ms
Heard unknown (201) @32576ms
```

